Course:

Object Oriented Programming

2nd Year preparatory Students, 2026-2027

Chapter 3

OOP pillars: Core Concepts

Part 4: Polymorphism

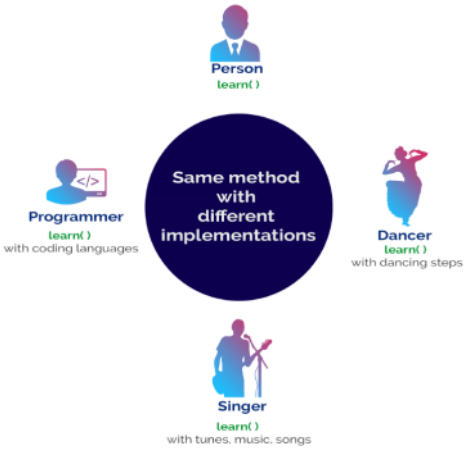
Keywords: Polymorphism concept, overriding, Overloading

Lecture notes prepared by Chamseddine Hamdeni.
Higher Institute of Medical Technologies of Tunis, Tunis El-Manar University
Chamseddine.hamdeni@istmt.utm.tn

1. Introduction

What is polymorphism ?

- Polymorphism contains two words "poly" and "morphs":
  ==> Poly means **many**, and morph means **shape**.
- By polymorphism, we understand that one task can be performed in different ways.
  Example: you have a class animal, and all animals speak. But they speak differently. Here, **the "speak" behavior is polymorphic and depends on the animal.**



In programming, Polymorphism is the ability of a variable, function, or object to take on multiple forms. For example, in a programming language that supports inheritance, classes in the same hierarchical tree may have methods **with the same name but different behaviors.**

Example

This code demonstrates the concept of inheritance and method **overriding** in Python classes. It shows **how subclasses can override methods defined in their parent class to provide specific behavior while still inheriting other methods** from the parent class

```
class Bird:

        def intro(self):
                print("There are many types of birds.")

        def flight(self):
                print("Most of the birds can fly but some cannot.")

class sparrow(Bird):

        def flight(self):
                print("Sparrows can fly.")

class ostrich(Bird): #نعامة

        def flight(self):
                print("Ostriches cannot fly.")

obj_bird = Bird()
obj_spr = sparrow()
obj_ost = ostrich()

obj_bird.intro()
obj_bird.flight()

obj_spr.intro()
obj_spr.flight()

obj_ost.intro()
obj_ost.flight()
```

2. Overriding

Method Signature

A method signature in object-oriented programming defines the structure of a method within a class. It includes the method **name**, the **number and type of parameters**, and the **return type**.

In Python, the method signature does not explicitly include the return type, since Python is a dynamically typed language. The method signature consists of the method name and its parameters.

For example, these two classes have 2 methods with the **same signatures**:

```
class class1:
    def method1(self):
        self.health -= 5
        return self.health
```

```
class class2:
    def method1(self):
        self.health -= 10
        return self.health
```

Both methods have the same name, take 0 inputs, and return integers. If any of those things are different, they would have different function signatures.

Here is an example of **different signatures**.

```
class class1:
    def method1(self):
        self.health -= 5
        return self.health
```

```
class class2:
    def method1(self, dmg):
        self.health -= dmg
        return self.health
```

Overriding

▁▂▄▅▆▇ 🎀 *d o n't mem rize definiti ns* 🎀 ▇▆▅▄▂▁

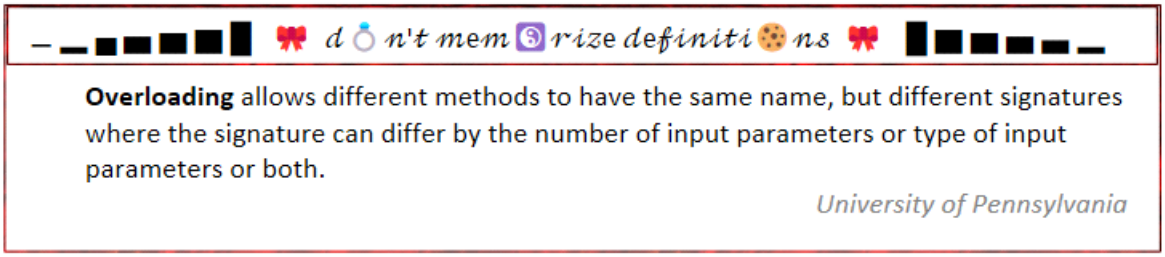
Overriding: When a subclass provides a specific implementation for a method that is already provided by its parent class, the method in the subclass is said to override the method in the superclass. When overriding a method, the name, return type, and arguments (i.e., the method **signature**) of the overriding method in the subclass must match exactly the signature of the overridden method in the superclass.

"Java: The Complete Reference" by Herbert Schildt

Example

```
class Bank:
    def getROI(self):
        return 10;
class BNA(Bank):
    def getROI(self):
        return 7;
class ZITOUNA(Bank):
    def getROI(self):
        return 8;
b1 = Bank()
b2 = BNA()
b3 = ZITOUNA()
print("Bank Rate of interest:",b1.getROI());
print("BNA Rate of interest:",b2.getROI());
print("ZITOUNA Rate of interest:",b3.getROI());
```

3. Overloading



Python does not support method overloading by default.
In contrast, languages like C++ and Java allow methods within a class to have the same name but different parameter lists (method overloading), and the correct method is chosen based on the method signature at compile time.

WHY ?
Python is **dynamically typed**, and the type of a function argument is determined at runtime;
Hence, the language does not need to differentiate between functions solely based on argument types.
In the **statically-typed** languages, the compiler can distinguish which method to invoke based on the number, type, and order of parameters at compile time.

Limitation: **Constructor Overloading is not supported in python**
Unlike other object-oriented languages, Python does not support constructor overloading. This means that you cannot have multiple constructors with different parameters in a single class.

GENERAL KNOWLEDGE

In French literature, original technical terms are frequently translated rather than retained:

- "Overriding" becomes "Redéfinition."
- "Overloading" becomes "Surchargement."

Overloading In Java

GENERAL KNOWLEDGE

```java
public class Processor {

    // Process an integer by squaring it
    public void process(int data) {
        System.out.println("Processing integer, square of " + data + ": " + (data * data));
    }

    // Process a string by reversing it
    public void process(String data) {
        String reversed = new StringBuilder(data).reverse().toString();
        System.out.println("Processing string, reversed: " + reversed);
    }

    public static void main(String[] args) {
        // Create an instance of Processor
        Processor p = new Processor();

        // Process an integer
        p.process(10);     // Outputs: Processing integer, square of 10: 100

        // Process a string
        p.process("Hello"); // Outputs: Processing string, reversed: olleH
    }
}
```

4. Exam Style Question

Context

Your task is to design a Python program that models different types of medical devices in a healthcare facility, such as thermometers and blood pressure monitors. The software should be flexible enough to accommodate different types of devices, and allow for easy addition of new types of devices in the future. **Use Abstraction and polymorphism to achieve that.**

Requirements

1- Create an **abstract class** named MedicalDevice with the following **abstract methods**:
   - read_data: To read data from the device.
   - device_type: To identify the type of the device.

2- Create a **concrete class** for a thermometer (Thermometer) that **inherits** from MedicalDevice. Implement the abstract methods in the following way:
   - read_data: Should return a random temperature between 97.0F and 99.5F.
   - device_type: Should return the string "Thermometer".

3- Create a **concrete class** for a blood pressure monitor (BloodPressureMonitor) that also **inherits** from MedicalDevice. Implement the abstract methods as follows:
   - read_data: Should return a random systolic and diastolic blood pressure within a typical range (e.g., systolic between 90 and 120, and diastolic between 60 and 80).
   - device_type: Should return the string "Blood Pressure Monitor".

4- Create another concrete class for a pulse oximeter (PulseOximeter). Implement the abstract methods like so:
   - read_data: Should return a random SpO2 level between 94% and 100%.
   - device_type: Should return the string "Pulse Oximeter".

5- Write a function named display_device_data that takes a **list** of MedicalDevice objects and prints out the type of each device along with the data it reads using a loop. **This function should demonstrate polymorphism.**

6- Write the Python code in the main program that performs the following:
   - Instantiates one object for each concrete class (Thermometer, BloodPressureMonitor, PulseOximeter).
   - Adds these objects to a list.
   - Calls the display_device_data() function, passing the list of objects as an argument.

Solution

```
import random
from abc import ABC, abstractmethod

# Abstract Class for Medical Device
class MedicalDevice(ABC):

    @abstractmethod
    def read_data(self):
        pass

    @abstractmethod
    def device_type(self):
        pass

# Concrete Class for Thermometer
class Thermometer(MedicalDevice):
    def read_data(self):
        return str(round(random.uniform(97.0, 99.5), 2)) + "F"
    def device_type(self):
        return "Thermometer"

# Concrete Class for Blood Pressure Monitor
class BloodPressureMonitor(MedicalDevice):
    def read_data(self):
        systolic = random.randint(90, 120)
        diastolic = random.randint(60, 80)
        return str(systolic) + "/" + str(diastolic)
    def device_type(self):
        return "Blood Pressure Monitor"

# Concrete Class for Pulse Oximeter
class PulseOximeter(MedicalDevice):
    def read_data(self):
        return "SpO2 " + str(random.randint(94, 100)) + "%"
    def device_type(self):
        return "Pulse Oximeter"

# Function to demonstrate polymorphism
def display_device_data(devices):
    for device in devices:
        print("Device Type:", device.device_type(), ", Data:", device.read_data())
# Instantiate objects
thermo = Thermometer()
bp_monitor = BloodPressureMonitor()
pulse_ox = PulseOximeter()
# List of different device types
devices = [thermo, bp_monitor, pulse_ox]
# Polymorphism in action: Display dynamic data for all devices
display_device_data(devices)
```